

Linear Regression

Key Notation:

- $\mathbf{X} \in \mathbb{R}^{N \times D}$ – Data matrix with N examples, D features
- $\mathbf{y} \in \mathbb{R}^N$ – Target values
- $\mathbf{w} \in \mathbb{R}^D$ – Weight vector (parameters)
- $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ – Predictions

Ordinary Least Squares:

$$L(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - \mathbf{w}^T \mathbf{x}_n)^2 \quad (1)$$

$$\nabla L(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^T \mathbf{x}_n)(-\mathbf{x}_n) = -\mathbf{X}^T \mathbf{y} + \mathbf{X}^T \mathbf{X} \mathbf{w} \quad (2)$$

Closed-form solution: $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

Regularization:

- **Ridge (L2):** $L(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - \mathbf{w}^T \mathbf{x}_n)^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$
- Solution: $\mathbf{w}_{ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$
- **Lasso (L1):** $L(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - \mathbf{w}^T \mathbf{x}_n)^2 + \lambda \|\mathbf{w}\|_1$

Bias-Variance Tradeoff:

$$\text{MSE} = \mathbb{E}_{D, y|x} [(y - f_D(x))^2] \quad (3)$$

$$= \mathbb{E}_{y|x} [(y - \bar{y})^2] + (\bar{y} - \bar{f}(x))^2 + \mathbb{E}_D [(\bar{f}(x) - f_D(x))^2] \quad (4)$$

$$= \text{noise}(x) + \text{bias}(f(x))^2 + \text{var}(f(x)) \quad (5)$$

Regularization effect: Increases bias, reduces variance

Classification

Loss Functions:

- **0/1 Loss:** $L(y, \hat{y}) = 1[y \neq \hat{y}]$ (non-differentiable)
- **Hinge Loss:** $L(y, f(x)) = \max(0, 1 - yf(x))$
- **Logistic Loss:** $L(y, f(x)) = \log(1 + e^{-yf(x)})$

Perceptron:

- Prediction: $\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x})$
- Update: $\mathbf{w} \leftarrow \mathbf{w} + \alpha y \mathbf{x}$ if misclassified
- Convergence guaranteed if data is linearly separable

Logistic Regression:

- Sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Probability: $p(y=1|\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{x})$
- Log-likelihood: $\ell(\mathbf{w}) = \sum_{n=1}^N [y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n)]$
- Gradient: $\nabla \ell(\mathbf{w}) = \sum_{n=1}^N (y_n - \hat{y}_n) \mathbf{x}_n$

Multi-class Logistic Regression:

- Softmax function: $\sigma_i(z) = \frac{e^{z_i}}{\sum_j e^{z_j}}$
- Probability: $p(y=i|\mathbf{x}, \mathbf{W}) = \frac{e^{\mathbf{w}_i^T \mathbf{x}}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x}}}$

Naïve Bayes (Generative):

- Bayes Rule: $p(y|\mathbf{x}) \propto p(\mathbf{x}|y)p(y)$
- Naïve assumption: $p(\mathbf{x}|y) = \prod_j p(x_j|y)$
- Class prior: $p(y = c_k) = \pi_k$ (MLE: proportion in training data)

Gradient Descent:

- Batch: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla L(\mathbf{w}_t)$
- Stochastic: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla L_n(\mathbf{w}_t)$

Neural Networks

Network Structure:

- Input layer: $\mathbf{x} \in \mathbb{R}^{d_0}$
- Hidden layer ℓ : $\mathbf{h}^{(\ell)} = g(\mathbf{W}^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)})$
- Output layer: $\hat{\mathbf{y}} = g(\mathbf{W}^{(L)} \mathbf{h}^{(L-1)} + \mathbf{b}^{(L)})$

Activation Functions:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$, range (0, 1)
- Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, range (-1, 1)
- ReLU: $\text{ReLU}(z) = \max(0, z)$

Backpropagation:

- Forward pass: Compute activations through the network
- Backward pass: Compute gradients using the chain rule
- For output layer: $\delta^{(L)} = \nabla_{\hat{\mathbf{y}}} L \odot g'(\mathbf{z}^{(L)})$
- For hidden layers: $\delta^{(\ell)} = ((\mathbf{W}^{(\ell+1)})^T \delta^{(\ell+1)}) \odot g'(\mathbf{z}^{(\ell)})$
- Weight gradients: $\nabla_{\mathbf{W}^{(\ell)}} L = \delta^{(\ell)} (\mathbf{h}^{(\ell-1)})^T$

Loss Functions:

- Regression: MSE Loss = $\frac{1}{2} \sum_{n=1}^N (\mathbf{y}_n - \hat{\mathbf{y}}_n)^2$
- Classification: Cross-Entropy = $-\sum_{n=1}^N \sum_{c=1}^C y_{nc} \log(\hat{y}_{nc})$

Model Selection:

- Cross-validation: Split data into K folds, train on K-1 and test on 1
- Hyperparameter tuning: Architecture, learning rate, regularization
- Regularization: Weight decay, dropout, early stopping

Support Vector Machines

Hard-Margin SVM:

- Decision boundary: $\mathbf{w}^T \mathbf{x} + w_0 = 0$
- Functional margin: $y_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1$
- Geometric margin: $\frac{\mathbf{w}^T \mathbf{x} + w_0}{\|\mathbf{w}\|_2}$
- Optimization problem: $\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|_2^2$ s.t. $y_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1, \forall n$

Soft-Margin SVM:

- Includes slack variables $\xi_n \geq 0$
- Optimization problem: $\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{n=1}^N \xi_n$
- Subject to: $y_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1 - \xi_n, \xi_n \geq 0, \forall n$
- Parameter C trades off margin size vs. training errors

Kernel Trick:

- Implicit feature mapping: $\phi: \mathcal{X} \rightarrow \mathcal{F}$
- Kernel function: $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z})$
- Decision function: $f(\mathbf{x}) = \sum_{n=1}^N \alpha_n y_n K(\mathbf{x}_n, \mathbf{x}) + w_0$

Common Kernels:

- Linear: $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$
- Polynomial: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$
- RBF/Gaussian: $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|_2^2)$

Support Vectors:

- Points with $\alpha_i > 0$ (on margin boundary or misclassified)
- Only support vectors affect decision boundary

Hidden Markov Models

Key Notation:

- $s_t \in \{0, 1\}^K$ – Hidden state at time t (one-hot encoded)
- x_t – Observation at time t
- $\theta \in [0, 1]^K$ – Initial state distribution, $\sum_k \theta_k = 1$
- $T \in [0, 1]^{K \times K}$ – Transition matrix, $T_{i,j} = P(s_{t+1} = j | s_t = i)$
- $\pi \in [0, 1]^{K \times M}$ – Emission parameters, $\pi_{k,m} = P(x_t = m | s_t = k)$

Forward-Backward Algorithm:

Forward Values (α): $\alpha_t(s_t) = p(x_1, \dots, x_t, s_t)$

Backward Values (β): $\beta_t(s_t) = p(x_{t+1}, \dots, x_n | s_t)$

Forward Algorithm:

- **Initialize:** For $t = 1$, all states $s_1 \in \{1, \dots, K\}$: $\alpha_1(s_1) = p(x_1 | s_1) p(s_1) = \pi_{s_1, x_1} \cdot \theta_{s_1}$
 - **Recursion:** For $t = 2, \dots, n$, all states $s_t \in \{1, \dots, K\}$: $\alpha_t(s_t) = p(x_t | s_t) \sum_{s_{t-1}} p(s_t | s_{t-1}) \alpha_{t-1}(s_{t-1})$
- Meaning: Take emission probability $p(x_t | s_t)$ and multiply by sum over all possible previous states. For each previous state, multiply transition probability $p(s_t | s_{t-1}) = T_{s_{t-1}, s_t}$ with previous forward value $\alpha_{t-1}(s_{t-1})$.

Backward Algorithm:

- **Initialize:** For $t = n$ (final time step), all states $s_n \in \{1, \dots, K\}$: $\beta_n(s_n) = 1$
 - **Recursion:** For $t = n-1, n-2, \dots, 1$, all states $s_t \in \{1, \dots, K\}$: $\beta_t(s_t) = \sum_{s_{t+1}} p(s_{t+1} | s_t) p(x_{t+1} | s_{t+1}) \beta_{t+1}(s_{t+1})$
- Meaning: Sum over all possible next states. For each next state, multiply transition probability $p(s_{t+1} | s_t) = T_{s_t, s_{t+1}}$, emission probability $p(x_{t+1} | s_{t+1}) = \pi_{s_{t+1}, x_{t+1}}$, and next backward value $\beta_{t+1}(s_{t+1})$.

Using α and β for Inference:

- **Filtering:** $p(s_t | x_1, \dots, x_t) = \frac{\alpha_t(s_t)}{\sum_{s'_t} \alpha_t(s'_t)}$
- **Smoothing:** $p(s_t | x_1, \dots, x_n) = \frac{\alpha_t(s_t) \beta_t(s_t)}{\sum_{s'_t} \alpha_t(s'_t) \beta_t(s'_t)}$
- **Likelihood:** $p(x_1, \dots, x_n) = \sum_{s_t} \alpha_t(s_t) \beta_t(s_t)$ (any t)
- **Transition probabilities:** $p(s_t, s_{t+1} | x_1, \dots, x_n) = \frac{\alpha_t(s_t) p(s_{t+1} | s_t) p(x_{t+1} | s_{t+1}) \beta_{t+1}(s_{t+1})}{p(x_1, \dots, x_n)}$

Viterbi Algorithm

Finding the Most Likely State Sequence:

Steps:

- **Initialize:** For $t = 1$, all states $s_1 \in \{1, \dots, K\}$:

$$\gamma_1(s_1) = p(x_1 | s_1) p(s_1) = \pi_{s_1, x_1} \cdot \theta_{s_1} \quad (6)$$

$$\psi_1(s_1) = 0 \text{ (no previous state)} \quad (7)$$

- **Recursion:** For $t = 2, \dots, n$, all states $s_t \in \{1, \dots, K\}$:

$$\gamma_t(s_t) = p(x_t | s_t) \max_{s_{t-1}} [p(s_t | s_{t-1}) \gamma_{t-1}(s_{t-1})] \quad (8)$$

$$\psi_t(s_t) = \arg \max_{s_{t-1}} [p(s_t | s_{t-1}) \gamma_{t-1}(s_{t-1})] \quad (9)$$

For each current state s_t , find previous state s_{t-1} maximizing probability of reaching s_t . Store both max probability $\gamma_t(s_t)$ and previous state $\psi_t(s_t)$.

- **Termination:** Find most likely final state:

$$s_n^* = \arg \max_{s_n} \gamma_n(s_n) \quad (10)$$

$$P^* = \max_{s_n} \gamma_n(s_n) \quad (11)$$

- **Backtracking:** Recover most likely state sequence: $s_t^* = \psi_{t+1}(s_{t+1}^*)$ for $t = n-1, n-2, \dots, 1$

Expectation-Maximization (EM) Algorithm

Key Notation:

- x_n – Data point n
- $z_n \in \{0, 1\}^K$ – Latent variable (one-hot) indicating component for x_n
- $\theta \in [0, 1]^K$ – Mixture weights, $\sum_k \theta_k = 1$
- π_k – Parameters of component k in mixture
- $q_n \in [0, 1]^K$ – "Soft" assignments of data point n to components
- $q_{n,k}$ – Probability that data point n belongs to component k

Objective: **Maximum Likelihood** $\log p(X|\theta, \pi)$ = $\sum_{n=1}^N \log \sum_{k=1}^K \theta_k p(x_n|z_n = C_k, \pi_k)$
Complete-Data Log-Likelihood: $\log p(X, Z|\theta, \pi)$ = $\sum_{n=1}^N \sum_{k=1}^K z_{n,k} [\log \theta_k + \log p(x_n|z_n = C_k, \pi_k)]$
Evidence Lower Bound (ELBO): $\text{ELBO}(\theta, \pi, q)$ = $\sum_{n=1}^N \sum_{k=1}^K q_{n,k} [\log \theta_k + \log p(x_n|z_n = C_k, \pi_k) - \log q_{n,k}]$

EM Algorithm:

- **Initialize** parameters $\theta^{(0)}$ and $\pi^{(0)}$ (often randomly)
- **E-step:** Compute posterior probabilities:

$$q_{n,k}^{(i)} = \frac{\theta_k^{(i)} p(x_n|z_n = C_k; \pi_k^{(i)})}{\sum_{j=1}^K \theta_j^{(i)} p(x_n|z_n = C_j; \pi_j^{(i)})} \quad (12)$$

- **M-step:** Update parameters:

$$\theta^{(i+1)}, \pi^{(i+1)} = \arg \max_{\theta, \pi} \sum_{n=1}^N \sum_{k=1}^K q_{n,k}^{(i)} [\log \theta_k + \log p(x_n|z_n = C_k, \pi_k)] \quad (13)$$

For mixture weights:

$$\theta_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)}}{N} \quad (14)$$

For component parameters:

$$\pi_k^{(i+1)} = \arg \max_{\pi_k} \sum_{n=1}^N q_{n,k}^{(i)} \log p(x_n|z_n = C_k, \pi_k) \quad (15)$$

- **Repeat** until convergence

Gaussian Mixture Model: E-step:

$$q_{n,k}^{(i)} = \frac{\theta_k^{(i)} \mathcal{N}(x_n; \mu_k^{(i)}, \Sigma_k^{(i)})}{\sum_{j=1}^K \theta_j^{(i)} \mathcal{N}(x_n; \mu_j^{(i)}, \Sigma_j^{(i)})} \quad (16)$$

M-step:

$$\mu_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)} x_n}{\sum_{n=1}^N q_{n,k}^{(i)}} \quad (17)$$

$$\Sigma_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)} (x_n - \mu_k^{(i+1)})(x_n - \mu_k^{(i+1)})^T}{\sum_{n=1}^N q_{n,k}^{(i)}} \quad (18)$$

Binary Data EM Example: E-step:

$$q_{n,k}^{(i)} = \frac{\theta_k^{(i)} \prod_{d=1}^D (\mu_{k,d}^{(i)})^{x_{n,d}} (1 - \mu_{k,d}^{(i)})^{1-x_{n,d}}}{\sum_{j=1}^K \theta_j^{(i)} \prod_{d=1}^D (\mu_{j,d}^{(i)})^{x_{n,d}} (1 - \mu_{j,d}^{(i)})^{1-x_{n,d}}} \quad (19)$$

M-step:

$$\theta_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)}}{N} \quad (20)$$

$$\mu_{k,d}^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)} x_{n,d}}{\sum_{n=1}^N q_{n,k}^{(i)}} \quad (21)$$

Principal Component Analysis (PCA)

Key Notation:

- $X \in \mathbb{R}^{N \times D}$ – Data matrix with N examples in D dimensions
- $\bar{X} \in \mathbb{R}^{1 \times D}$ – Mean of each feature across all examples
- $X' = X - \mathbf{1}\bar{X}$ – Centered data matrix
- $S = \frac{1}{N} X'^T X'$ – Empirical covariance matrix
- $v_1, v_2, \dots, v_D \in \mathbb{R}^D$ – Eigenvectors of S (principal components)
- $\lambda_1, \lambda_2, \dots, \lambda_D$ – Eigenvalues (variances along PCs)
- $W = [v_1, v_2, \dots, v_{D'}]$ – Projection matrix for dimension reduction
- $Z = X'W$ – Projected data in lower dimension D'

Understanding the Covariance Matrix:

- $S = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T$
- Diagonal elements S_{ii} = variance of feature i
- Off-diagonal elements S_{ij} = covariance between features i and j
- Symmetric: $S_{ij} = S_{ji}$
- Positive semi-definite: $v^T S v \geq 0$ for any vector v
- Each element: $S_{ij} = \frac{1}{N} \sum_{n=1}^N (x_{ni} - \bar{x}_i)(x_{nj} - \bar{x}_j)$

Finding Eigenvectors and Eigenvalues: An eigenvector v of matrix S satisfies: $Sv = \lambda v$

- Find characteristic equation: $\det(S - \lambda I) = 0$
- Solve for roots to find eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_D$
- For each λ_i , find eigenvector v_i by solving: $(S - \lambda_i I)v_i = 0$
- Normalize: $v_i := \frac{\bar{v}_i}{\|\bar{v}_i\|}$
- Sort eigenvectors by descending eigenvalues

PCA Algorithm:

- **Center the Data:** $\bar{x}_d = \frac{1}{N} \sum_{n=1}^N x_{nd}$, $x'_{nd} = x_{nd} - \bar{x}_d$, $X' = X - \mathbf{1}\bar{X}$
- **Compute Covariance Matrix:** $S = \frac{1}{N} X'^T X' = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T$
- **Find Eigenvectors and Eigenvalues:** Decompose $S = V\Lambda V^T$ where V has eigenvectors as columns
- **Sort and Select:** Reorder so $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D$. Select first D' eigenvectors: $W = [v_1, v_2, \dots, v_{D'}]$

Variance explained: $\frac{\sum_{i=1}^{D'} \lambda_i}{\sum_{i=1}^D \lambda_i}$

- **Project the Data:** $Z = X'W$

Two Views of PCA:

- **Variance Maximization:** PCA finds directions of maximum variance $v_1 = \arg \max_{\|v\|=1} v^T S v$
- **Reconstruction Error Minimization:** PCA minimizes squared reconstruction error $\min_{W: W^T W = I} \|X' - X'WW^T\|_F^2$

PCA with SVD: $X' = USV^T$ where:

- U is $N \times N$ orthogonal
- S is $N \times D$ diagonal with singular values
- V is $D \times D$ orthogonal
- Columns of V are eigenvectors of $X'^T X'$

- Singular values relate to eigenvalues: $\lambda_i = \frac{s_i^2}{N}$
- Projected data: $Z = X'V_{:,D'}$ or $Z = US_{:,D'}$

K-Means Clustering

Key Notation:

- $\{x_n\}_{n=1}^N$ – Data points to be clustered
- $\{\mu_c\}_{c=1}^C$ – Cluster centers (centroids)
- $r_{nc} \in \{0, 1\}$ – Binary indicator: 1 if point n is assigned to cluster c , 0 otherwise

Objective Function: $L(X, \{\mu_c\}_{c=1}^C, \{r_{nc}\}) = \sum_{n=1}^N \sum_{c=1}^C r_{nc} \|x_n - \mu_c\|_2^2$

Lloyd's Algorithm (K-Means):

- **Initialize** cluster centers $\mu_1, \mu_2, \dots, \mu_K$:
 - Random initialization: Select K random data points
 - K-means++ initialization: Select centers far apart
 - * Choose first center randomly from data
 - * For remaining centers, choose data points with probability proportional to squared distance from nearest existing center

- **Assignment step (E-step):** Assign each data point to closest center: $r_{nc} = \begin{cases} 1 & \text{if } c = \arg \min_{c'} \|x_n - \mu_{c'}\|^2 \\ 0 & \text{otherwise} \end{cases}$
 - **Update step (M-step):** Update centers to be mean of assigned points: $\mu_c = \frac{\sum_{n=1}^N r_{nc} x_n}{\sum_{n=1}^N r_{nc}}$
 - **Repeat** until convergence (assignments stop changing or centers move less than threshold)
- Practical Considerations:**
- **Choosing K :** "Elbow method" plots objective function vs. different K values and looks for "elbow" where improvement slows
 - **Multiple restarts:** Run K-Means multiple times with different initializations and select best result
 - **Standardization:** Standardize features to zero mean and unit variance
 - **Handling categorical data:** Use K-Medoids instead, which uses actual data points as centers and allows arbitrary distance metrics

Mixture Models

Key Notation:

- θ_k – Mixture weight for component k , $\sum_{k=1}^K \theta_k = 1$
- π_k – Parameters of component k
- z_n – Latent variable indicating which component generated data point x_n

Mixture Model Formulation: $p(x; \{\pi_k\}_{k=1}^K, \theta) = \sum_{k=1}^K \theta_k p_{\text{class}}(x|z = C_k; \pi_k)$

Gaussian Mixture Model (GMM):

$$p(x|z = C_k; \mu_k, \Sigma_k) = \mathcal{N}(x; \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} |\Sigma_k|^{1/2}} e^{-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)} \quad (22)$$

Full model: $p(x; \{\mu_k, \Sigma_k\}_{k=1}^K, \theta) = \sum_{k=1}^K \theta_k \mathcal{N}(x; \mu_k, \Sigma_k)$

EM Algorithm for GMM:

- **Initialize** $\theta^{(0)}, \{\mu_k^{(0)}, \Sigma_k^{(0)}\}_{k=1}^K$:
 - Often use K-Means for initial centers $\mu_k^{(0)}$
 - Set $\Sigma_k^{(0)}$ to sample covariance or identity
 - Set $\theta_k^{(0)} = 1/K$ (uniform)
- **E-step:** Compute posterior probabilities: $q_{n,k}^{(i)} = \frac{\theta_k^{(i)} \mathcal{N}(x_n; \mu_k^{(i)}, \Sigma_k^{(i)})}{\sum_{j=1}^K \theta_j^{(i)} \mathcal{N}(x_n; \mu_j^{(i)}, \Sigma_j^{(i)})}$
- **M-step:** Update parameters:

$$\theta_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)}}{N} \quad (23)$$

$$\mu_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)} x_n}{\sum_{n=1}^N q_{n,k}^{(i)}} \quad (24)$$

$$\Sigma_k^{(i+1)} = \frac{\sum_{n=1}^N q_{n,k}^{(i)} (x_n - \mu_k^{(i+1)})(x_n - \mu_k^{(i+1)})^T}{\sum_{n=1}^N q_{n,k}^{(i)}} \quad (25)$$

- **Repeat** until convergence (log-likelihood stops increasing significantly)

Connection between K-Means and GMM:

- K-Means is a special case of GMM with constraints:
- All covariance matrices are ϵI (spherical with same tiny variance)
- Hard assignments instead of soft probabilities
- As $\epsilon \rightarrow 0$, GMM with EM approaches K-Means

Reinforcement Learning

Key Notation:

- S – Set of states
- A – Set of actions
- $p(s'|s, a)$ – Transition function
- $r(s, a)$ – Reward function
- $\gamma \in [0, 1]$ – Discount factor

- $\pi: S \rightarrow A$ – Policy
- $V^\pi(s)$ – Value function under policy π
- $Q^\pi(s, a)$ – Action-value function under policy π

Q-Learning Algorithm:

- **Initialize** $Q(s, a)$ arbitrarily for all state-action pairs:
 - Often zeros or small random values
 - Optimistic initialization (high values) encourages exploration
- **For each episode:**
 - **Initialize** starting state s
 - **For each step:**
 - * **Choose action** a from state s using policy from Q :
 - ϵ -greedy: With probability ϵ , choose random action; otherwise, $a = \arg \max_{a'} Q(s, a')$
 - Reduce ϵ over time
 - * **Take action** a , observe reward r and next state s'
 - * **Update** $Q(s, a): Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ where:
 - $\alpha \in (0, 1]$ is the learning rate
 - $r + \gamma \max_{a'} Q(s', a')$ is the TD target
 - $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ is the TD error
 - * **Update state:** $s \leftarrow s'$
 - **Continue** until s is terminal or maximum steps reached

SARSA Algorithm:

- **Initialize** $Q(s, a)$ arbitrarily for all state-action pairs
- **For each episode:**
 - **Initialize** starting state s
 - **Choose action** a from state s using policy from Q
 - **For each step:**
 - * **Take action** a , observe reward r and next state s'
 - * **Choose action** a' from state s' using policy from Q
 - * **Update** $Q(s, a): Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
 - * **Update state and action:** $s \leftarrow s', a \leftarrow a'$

Key Differences between Q-Learning and SARSA:

- **Target Policy:**
 - Q-Learning: Uses $\max_{a'} Q(s', a')$ (optimal action)
 - SARSA: Uses $Q(s', a')$ (next action according to current policy)
- **Policy Type:**
 - Q-Learning: Off-policy (learns optimal policy while following exploration policy)
 - SARSA: On-policy (learns about the policy it's following)
- **Exploration-Exploitation:**
 - Q-Learning: More aggressive (can learn optimal policy with random exploration)
 - SARSA: More conservative (considers exploration in updates)

Exploration vs. Exploitation:

- **Exploration:** Trying new actions to discover better strategies
 - **Exploitation:** Selecting actions known to yield good rewards
- Common exploration strategies:
- **ϵ -greedy:** Random action with probability ϵ , best action with $1 - \epsilon$
 - **Softmax:** Choose actions with probability proportional to estimated values
 - **Optimistic initialization:** Initialize Q-values high to encourage exploration
 - **UCB:** Select actions maximizing $Q(s, a) + c\sqrt{\frac{\ln(t)}{N(s, a)}}$

Time Complexity Analysis

Common Algorithm Complexities:

- **Linear Regression:** $O(Nd^2 + d^3)$ for optimal solution
- **Gradient Descent:** $O(Ndt)$ for t iterations
- **Forward-Backward (HMM):** $O(TK^2)$ for sequence length T , K states
- **Viterbi Algorithm:** $O(TK^2)$ for sequence length T , K states
- **EM Algorithm (GMM):** $O(NKDT)$ for N data points, K components, D dimensions, T iterations
- **PCA:** $O(ND^2 + D^3)$ for computation, dominated by covariance matrix and eigendecomposition
- **K-means:** $O(NKDT)$ for N data points, K clusters, D dimensions,

- T iterations
- **Q-Learning/SARSA:** $O(|S||A|T)$ for $|S|$ states, $|A|$ actions, T time steps
- **SVMs:** $O(N^2 \text{ to } N^3)$ depending on implementation
- **Neural Networks:** $O(N_\ell \cdot \sum_\ell n_\ell n_{\ell-1})$ for training

Detailed Breakdown:

HMM Forward Algorithm:

- Initialization: $O(K)$ operations
- For each time step t (T steps):
 - For each state s_t (K states):
 - * Sum over all previous states s_{t-1} (K states): $O(K)$
- Total: $O(TK^2)$

K-means:

- For each iteration (T iterations):
 - For each point (N points):
 - * Calculate distance to each centroid (K centroids):
 - Each distance calculation: $O(D)$ operations
 - Update K centroids: $O(ND)$ operations
 - Total: $O(TNKD)$

PCA:

- Computing covariance matrix: $O(ND^2)$
- Eigendecomposition of $D \times D$ matrix: $O(D^3)$
- Projection: $O(ND')$ where D' is reduced dimension
- Total: $O(ND^2 + D^3)$

EM for GMM:

- For each iteration (T iterations):
 - E-step: For each data point (N points) and component (K components):
 - * Computing Gaussian density: $O(D^2)$ for each
 - M-step: Update parameters: $O(NKD^2)$
- Total: $O(TNKD^2)$